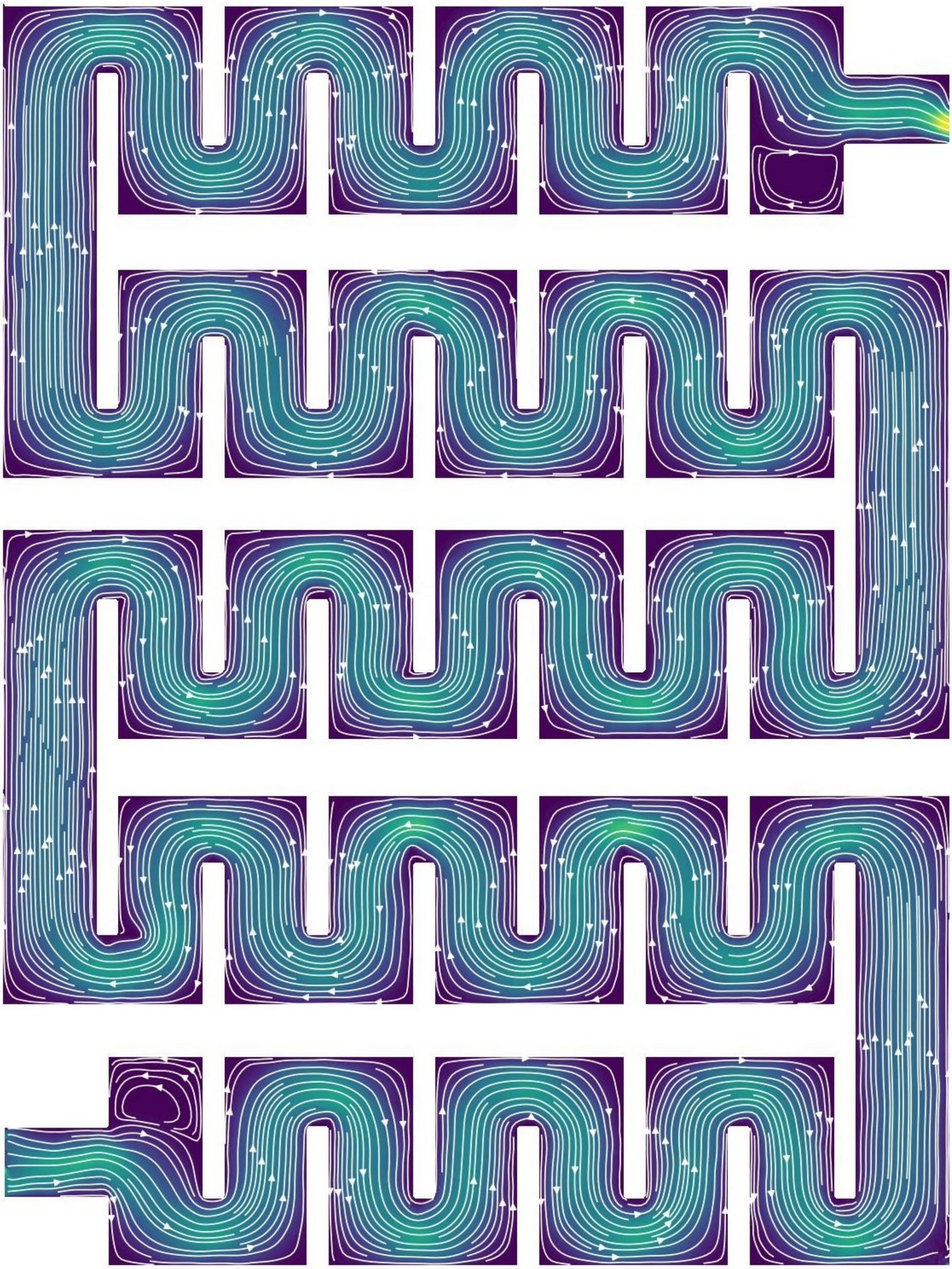
Finite Element Method Applied to Stokes Equations

Oleksandr Samoliuk

Summer Semester, 2026

Abstract

Derivations for Stokes Equations



Contents

1 General Information

Stokes flow, also named creeping flow or creeping motion, is a type of fluid flow where advective inertial forces are negligibly small compared with viscous forces. This regime is mathematically characterized by a low Reynolds number, i.e.,

$$\mathrm{Re} = \frac{\rho U L}{\mu} \ll 1 \quad \nu = \frac{\mu}{\rho},$$

where ρ is the fluid density, U is a characteristic velocity scale, L is a characteristic length scale, μ is the dynamic viscosity, and ν is the kinematic viscosity. This situation is highly typical in applications where the fluid velocities are remarkably slow, the fluid viscosities are exceptionally large, or the geometric length-scales of the flow domain are micro- or nano-sized.

Creeping flow was historically first analyzed by Sir George Gabriel Stokes to model the settling of spheres and understand hydrodynamic lubrication. In nature, this type of flow governs the swimming mechanics of microorganisms, the movement of lava, and the slow mantle convection currents deep within the Earth. In engineering and biotechnology, it occurs prominently in painting processes, micro-electromechanical systems (MEMS), microfluidic lab-on-a-chip devices, and the transport of viscous polymers during extrusion.

1.1 Mathematical Characterization and Challenges

From a mathematical perspective, the Stokes equations are obtained by simplifying the non-linear incompressible Navier-Stokes equations:

$$\begin{cases} \dfrac{\partial \mathbf{u}}{\partial t} + (\mathbf{u}\cdot\nabla)\mathbf{u} = -\dfrac{1}{\rho}\nabla p + \nu\nabla^2\mathbf{u} + \mathbf{f} \\ \nabla\cdot\mathbf{u} = 0 \end{cases}$$

Where:

- $\mathbf{u}$ is the flow velocity vector field.
- t represents time.
- p is the scalar static pressure field.
- ρ is the constant fluid density.
- ν is the kinematic viscosity.
- $\mathbf{f}$ represents external body force accelerations (such as gravity).

By dropping the convective inertial acceleration term $(\mathbf{u}\cdot\nabla)\mathbf{u}$, the momentum equation can be linearized, transforming into a system where the velocity field $\mathbf{u}$ and the pressure field p are coupled strictly through viscous dissipation and incompressibility constraints.

The absence of a time-derivative or non-linear term simplifies some analytical aspects but introduces profound structural numerical challenges for the Finite Element Method:

- **Saddle-Point Structure:** The incompressibility constraint $\nabla\cdot\mathbf{u} = 0$ acts as a structural constraint on the velocity space, giving the weak formulation a classic *saddle-point* (or constrained optimization) algebraic structure.
- **The Inf-Sup (LBB) Condition:** Unlike standard elliptic systems (such as the Poisson equation), one cannot arbitrarily pair discrete finite element approximation spaces for velocity (X_h) and pressure (Q_h). Unstable combinations lead to spurious numerical pressure oscillations ("checkerboard" pressure fields). To guarantee stability and convergence, the discrete spaces must satisfy the strict *Ladyzhenskaya-Babuška-Brezzi (LBB)* or *inf-sup* condition, requiring deliberate pairings such as the Taylor-Hood element (P_k/P_{k-1}) or stabilized formulations.

1.2 Stable Macro-Element Discretizations: The P_1-iso-P_1 Element

To circumvent the structural instabilities imposed by the inf-sup condition without resorting to complex higher-order polynomial expansions or non-conforming spaces, mixed finite element frameworks can leverage hierarchical macro-element structures. A prominent and computationally efficient choice within this class is the P_1-iso-P_1 element pair, historically introduced by Bercovier and Pironneau.

Instead of varying the polynomial degree between variables on a single mesh, the P_1-iso-P_1 method varies the underlying triangulation density while keeping both velocity and pressure spaces continuously piecewise linear (P_1). The discretization relies on a coarse macro-mesh $\mathcal{T}_H$ and a fine mesh $\mathcal{T}_h$. The fine mesh is constructed via a uniform refinement protocol where each parent macro-triangle $K \in \mathcal{T}_H$ is subdivided into four congruent child triangles by connecting the midpoints of its three topological edges, effectively halving the characteristic mesh size such that $h = H/2$.

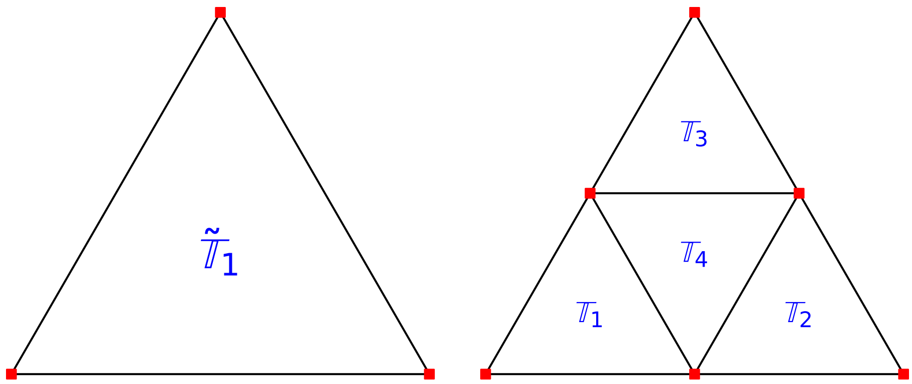
The discrete spaces for the fluid variables are then defined as:

$$X_h = \{\mathbf{v}_h \in C^0(\overline{\Omega})^2 : \mathbf{v}_h|_K \in P_1(K)^2, \forall K \in \mathcal{T}_h\}$$
$$Q_h = \{q_h \in C^0(\overline{\Omega}) : q_h|_M \in P_1(M), \forall M \in \mathcal{T}_H\}$$

By defining the velocity field $\mathbf{u}_h$ on the refined grid $\mathcal{T}_h$ and the pressure field p_h on the coarse macro-grid $\mathcal{T}_H$, the velocity space is enriched with a substantial number of internal degrees of freedom. This spatial scale separation provides a stable velocity-to-pressure ratio that satisfies the discrete Ladyzhenskaya-Babuška-Brezzi (LBB) condition:

$$\exists \beta > 0 : \quad \inf_{q \in Q_{\setminus\{0\}}} \sup_{v \in V_{\setminus\{0\}}} \left(\frac{\int_\Omega q(\nabla \cdot v) d\Omega}{\|v\|_V \cdot \|q\|_Q} \right) \geq \beta$$

where the stability constant $\beta > 0$ remains strictly bounded away from zero independently of the mesh refinement level. This macro-element architecture yields a highly symmetric structural layout, allowing the numerical engine to enjoy the low computational overhead of linear shape functions while retaining robust convergence properties.



2 Strong Formulation

The steady-state behavior of an incompressible, highly viscous fluid where inertial forces are negligible relative to viscous friction is governed by the classical Stokes equations. This system of partial differential equations forms a fundamental linear model in fluid mechanics. Given a bounded domain $\Omega \subset \mathbb{R}^2$ enclosed by a boundary $\partial\Omega = \Gamma_D \cup \Gamma_N$ (where $\Gamma_D \cap \Gamma_N = \emptyset$), the strong formulation of the problem seeks the velocity field $\mathbf{u}(x, y)$ and the scalar pressure field $p(x, y)$ satisfying:

$$\begin{cases} -\nu\Delta u + \nabla p = 0 & \text{in } \Omega, \\ \nabla \cdot u = 0 & \text{in } \Omega, \\ u = g & \text{on } \Gamma_D, \\ -p\, n + \nu(\nabla u)n = h & \text{on } \Gamma_N. \end{cases}$$

The mathematical and physical roles of these individual components are summarized as follows:

- **Momentum Balance ($-\nu\Delta\mathbf{u}+\nabla p = \mathbf{0}$):** This represents the conservation of momentum. It dictates a strict equilibrium between the internal viscous shear stresses—scaled by the kinematic viscosity ν—and the driving force generated by the pressure gradient ∇p.
- **Incompressibility Constraint ($\nabla \cdot \mathbf{u} = 0$):** This serves as the conservation of mass equation for an incompressible fluid. It requires the velocity field to be divergence-free, ensuring that fluid volume is perfectly preserved at every point within the domain.
- **Dirichlet Boundary Condition ($\mathbf{u} = \mathbf{g}$):** Enforced on the boundary segment Γ_D, this prescribes the exact fluid velocity. It is commonly used to dictate non-zero flow profiles at fluid inlets or to impose the physical "no-slip" condition ($\mathbf{u} = \mathbf{0}$) along stationary solid walls.
- **Neumann Boundary Condition ($-p\mathbf{n}+\nu(\nabla\mathbf{u})\mathbf{n} = \mathbf{h}$):** Enforced on the boundary segment Γ_N with outward unit normal vector $\mathbf{n}$, this specifies the weak normal stress or traction force vector $\mathbf{h}$ acting on the fluid, typically representing free-pressure outlets or open boundaries.

3 Weak Formulation

To solve the Stokes system using the Finite Element Method, we transform the original strong partial differential equations into their weak (or variational) integral form. This is achieved by multiplying the momentum and continuity equations by smooth test functions, $\mathbf{v}$ and q, and integrating the terms over the entire physical domain Ω. Applying integration by parts (Green's identity) to the second-order viscous diffusion term reduces the continuity requirements on our velocity fields from second-order derivatives down to first-order derivatives.

During this mathematical transition, the natural boundary conditions—such as the traction force h on the Neumann boundary Γ_N—automatically emerge within the boundary integrals. To accommodate non-zero (inhomogeneous) Dirichlet velocity profiles on the walls while strictly maintaining a homogeneous test function space X where $\mathbf{v} = \mathbf{0}$ on Γ_D, we apply a boundary lifting technique. We decompose the true velocity field into $\mathbf{u}_{total} = \mathbf{u} + \mathbf{r}_g$, where $\mathbf{r}_g$ is a known extension function matching the boundary data, and $\mathbf{u} \in X$ is the remaining homogeneous unknown field. This systematic process yields the coupled weak formulation system:

$$\begin{cases} \displaystyle\int_\Omega \nu \nabla u : \nabla v \, d\Omega - \int_\Omega p \, (\nabla \cdot v) \, d\Omega = \int_{\Gamma_N} h \cdot v \, d\Gamma - \int_\Omega \nu \nabla r_g : \nabla v \, d\Omega, & \forall v \in X, \\ \displaystyle -\int_\Omega q \, (\nabla \cdot u) \, d\Omega = \int_\Omega q \, (\nabla \cdot r_g) \, d\Omega, & \forall q \in Q. \end{cases} \tag{1}$$

4 Finite Element Discretization and Matrix Assembly

In this section, we transition from the continuous **weak formulation** to a **discrete algebraic representation** suitable for computational execution. By projecting the infinite-dimensional solution space onto a finite-dimensional subspace spanned by piecewise linear basis functions, we transform the governing integral operators into global sparse matrices. The following subsections detail the systematic assembly of these components.

4.1 Reference Triangle

To compute integrals efficiently, we map every arbitrary triangle in our mesh onto a standard master element called the reference triangle $\hat{T}$ in local coordinates (ξ_1, ξ_2). Its vertices are fixed at $\hat{\mathbf{x}}_1 = (0,0)$, $\hat{\mathbf{x}}_2 = (1,0)$, and $\hat{\mathbf{x}}_3 = (0,1)$. On this master domain, we define the standard linear Lagrange shape functions $\hat{\phi}_i(\xi_1, \xi_2)$:

$$
\begin{aligned}
&\hat{\phi}_1(\xi_1,\xi_2) = 1 - \xi_1 - \xi_2 \qquad && \hat{\phi}_1(0,0) = 1, \quad \hat{\phi}_1(1,0) = 0, \quad \hat{\phi}_1(0,1) = 0 \\
&\hat{\phi}_2(\xi_1,\xi_2) = \xi_1 && \hat{\phi}_2(0,0) = 0, \quad \hat{\phi}_2(1,0) = 1, \quad \hat{\phi}_2(0,1) = 0 \\
&\hat{\phi}_3(\xi_1,\xi_2) = \xi_2 && \hat{\phi}_3(0,0) = 0, \quad \hat{\phi}_3(1,0) = 0, \quad \hat{\phi}_3(0,1) = 1
\end{aligned}
$$

We will also take the gradients for easier future derivations and implementations:

$$
\nabla\hat{\phi}_1(\xi_1,\xi_2) = \begin{bmatrix} -1 \\ -1 \end{bmatrix} \qquad \nabla\hat{\phi}_2(\xi_1,\xi_2) = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \qquad \nabla\hat{\phi}_3(\xi_1,\xi_2) = \begin{bmatrix} 0 \\ 1 \end{bmatrix}
$$

We will now take a close look at the mapping and derive a Jacobian:

$$
x(\xi_1,\xi_2) = x_1 + (x_2 - x_1)\xi_1 + (x_3 - x_1)\xi_2
$$

$$
y(\xi_1,\xi_2) = y_1 + (y_2 - y_1)\xi_1 + (y_3 - y_1)\xi_2
$$

The mapping can be easily checked:

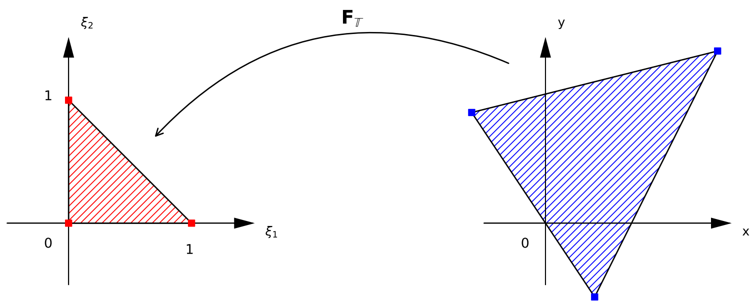
$$
\begin{aligned}
&x(0,0) = x_1 \qquad && x(1,0) = x_2 \qquad && x(0,1) = x_3 \\
&y(0,0) = y_1 && y(1,0) = y_2 && y(0,1) = y_3
\end{aligned}
$$

We can now take the Jacobian:

$$
\mathbf{J}_T = \begin{bmatrix} \frac{\partial x}{\partial \xi_1} & \frac{\partial x}{\partial \xi_2} \\ \frac{\partial y}{\partial \xi_1} & \frac{\partial y}{\partial \xi_2} \end{bmatrix} = \begin{bmatrix} x_2 - x_1 & x_3 - x_1 \\ y_2 - y_1 & y_3 - y_1 \end{bmatrix}
$$

It is also necessary to take a look at the original shape functions' derivatives:

$$
\begin{aligned}
\nabla\phi_i(x,y) = \begin{bmatrix} \frac{\partial \phi_i}{\partial x} \\ \frac{\partial \phi_i}{\partial y} \end{bmatrix} = \mathbf{J}_T^{-T} \cdot \nabla\hat{\phi}_i(\xi_1,\xi_2) = \begin{bmatrix} x_2 - x_1 & x_3 - x_1 \\ y_2 - y_1 & y_3 - y_1 \end{bmatrix}^{-T} \cdot \begin{bmatrix} \frac{\partial \hat{\phi}_i}{\partial \xi_1} \\ \frac{\partial \hat{\phi}_i}{\partial \xi_2} \end{bmatrix} = \\
= \left| \mathbf{X} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, \quad \mathbf{X}^{-1} = \frac{1}{ad - cb} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} \right| = \frac{1}{|\mathbf{J}_T|} \begin{bmatrix} y_3 - y_1 & y_1 - y_2 \\ x_1 - x_3 & x_2 - x_1 \end{bmatrix} \cdot \begin{bmatrix} \frac{\partial \hat{\phi}_i}{\partial \xi_1} \\ \frac{\partial \hat{\phi}_i}{\partial \xi_2} \end{bmatrix} = \\
= |\mathbf{J}_T|^{-1} \begin{bmatrix} (y_3 - y_1)\frac{\partial \hat{\phi}_i}{\partial \xi_1} - (y_2 - y_1)\frac{\partial \hat{\phi}_i}{\partial \xi_2} \\ -(x_3 - x_1)\frac{\partial \hat{\phi}_i}{\partial \xi_1} + (x_2 - x_1)\frac{\partial \hat{\phi}_i}{\partial \xi_2} \end{bmatrix}
\end{aligned}
\tag{2}
$$



4.2 Stiffness Matrix A

To determine the discrete operator for the viscous term, we begin with the bilinear form involving the velocity test and trial functions:

$$\boxed{\int_\Omega \nu \nabla u : \nabla v \, d\Omega}$$

$$\int_\Omega \nu \nabla \mathbf{u}(x,y) : \nabla \mathbf{v}(x,y) \, d\Omega = \left| \begin{matrix} \mathbf{u}(x,y) = \begin{bmatrix} u_x(x,y) \\ u_y(x,y) \end{bmatrix} \\ \\ \mathbf{v}(x,y) = \begin{bmatrix} v_x(x,y) \\ v_y(x,y) \end{bmatrix} \end{matrix} \right| = \int_\Omega \nu \left(\nabla u_x \cdot \nabla v_x + \nabla u_y \cdot \nabla v_y \right) \, d\Omega =$$

$$= \int_\Omega \nu \nabla u_x \cdot \nabla v_x \, d\Omega + \int_\Omega \nu \nabla u_y \cdot \nabla v_y \, d\Omega = \left| \begin{matrix} u_x(x,y) \approx \sum_{i=1}^{N_v} u_i^x \phi_i(x,y) \\ \\ u_y(x,y) \approx \sum_{i=1}^{N_v} u_i^y \phi_i(x,y) \end{matrix} \right| =$$

$$= \int_\Omega \nu \nabla \left(\sum_{i=1}^{N_v} u_i^x \phi_i \right) \cdot \nabla v_x \, d\Omega + \int_\Omega \nu \nabla \left(\sum_{i=1}^{N_v} u_i^y \phi_i \right) \cdot \nabla v_y \, d\Omega =$$

$$= \left[\sum_{i=1}^{N_v} u_i^x \cdot \int_\Omega \nu \nabla \phi_i \cdot \nabla v_x \, d\Omega \right] + \left[\sum_{i=1}^{N_v} u_i^y \cdot \int_\Omega \nu \nabla \phi_i \cdot \nabla v_y \, d\Omega \right] =$$

$$= \left| \begin{matrix} \text{We now set } v_x(x,y) = \phi_j(x,y) \\ \text{and } v_y(x,y) = \phi_j(x,y) \end{matrix} \right| =$$

$$= \left[\sum_{i=1}^{N_v} u_i^x \cdot \int_\Omega \nu \nabla \phi_i \cdot \nabla \phi_j \, d\Omega \right] + \left[\sum_{i=1}^{N_v} u_i^y \cdot \int_\Omega \nu \nabla \phi_i \cdot \nabla \phi_j \, d\Omega \right] =$$

$$= \sum_{i=1}^{N_v} A_{ji} u_i^x + \sum_{i=1}^{N_v} A_{ji} u_i^y = \mathbf{A}^T \mathbf{u}_x + \mathbf{A}^T \mathbf{u}_y = \left| \mathbf{A} = \mathbf{A}^T \right| =$$

$$= \mathbf{A}\mathbf{u}_x + \mathbf{A}\mathbf{u}_y = \begin{bmatrix} \mathbf{A} & \mathbf{A} \end{bmatrix} \begin{bmatrix} \mathbf{u}_x \\ \mathbf{u}_y \end{bmatrix}$$

Now using the derived coordinate transformation, performed with the help of the Jacobian, we can calculate the local matrix $\mathbf{A}^{loc} \in \mathbb{R}^{3\times3}$

$$\mathbf{A}_{ij} = \int_{\Omega} \nu \nabla\phi_i(x,y) \cdot \nabla\phi_j(x,y)\, d\Omega$$

$$\mathbf{A}_{ij}^{loc} = \left| \begin{array}{c} \nabla\phi(x,y) = \mathbf{J}_T^{-T} \nabla\hat{\phi}(\xi_1,\xi_2) \\ d\Omega = dxdy = |\mathbf{J}_T|\, d\xi_1 d\xi_2 \end{array} \right| =$$

$$= \int_{\hat{T}} \nu \left(\mathbf{J}_T^{-T}\nabla\hat{\phi}_i(\xi_1,\xi_2)\right)^T \left(\mathbf{J}_T^{-T}\nabla\hat{\phi}_j(\xi_1,\xi_2)\right) |\mathbf{J}_T|\; d\xi_1 d\xi_2 = \int_{\hat{T}} \nu \left(\nabla\hat{\phi}_i\right)^T \mathbf{J}_T^{-1}\mathbf{J}_T^{-T} \left(\nabla\hat{\phi}_j\right) |\mathbf{J}_T|\; d\xi_1 d\xi_2 =$$

$$= \int_{\hat{T}} \frac{\nu}{2\,|\mathbf{J}_T|} \left(\nabla\hat{\phi}_i\right)^T \begin{bmatrix} y_3 - y_1 & -(x_3 - x_1) \\ -(y_2 - y_1) & x_2 - x_1 \end{bmatrix} \begin{bmatrix} y_3 - y_1 & -(y_2 - y_1) \\ -(x_3 - x_1) & x_2 - x_1 \end{bmatrix} \left(\nabla\hat{\phi}_j\right) |\mathbf{J}_T|\; d\xi_1 d\xi_2 =$$

$$= \frac{\nu}{|\mathbf{J}_T|} \int_{\hat{T}} \left(\nabla\hat{\phi}_i\right)^T \mathbf{Q} \left(\nabla\hat{\phi}_j\right)\; d\xi_1 d\xi_2 = \frac{\nu}{2\,|\mathbf{J}_T|} \left(\nabla\hat{\phi}_i\right)^T \mathbf{Q} \left(\nabla\hat{\phi}_j\right)$$

After calculating the vector-matrix-vector products explicitly using the constant reference gradients $\nabla\hat{\phi}_1 = [-1,-1]^T$, $\nabla\hat{\phi}_2 = [1,0]^T$, and $\nabla\hat{\phi}_3 = [0,1]^T$, we obtain the fully expanded local element matrix:

$$\mathbf{A}^{loc} = \frac{\nu}{|\mathbf{J}_T|} \begin{bmatrix} \nabla\hat{\phi}_1\mathbf{Q}\nabla\hat{\phi}_1 & \nabla\hat{\phi}_1\mathbf{Q}\nabla\hat{\phi}_2 & \nabla\hat{\phi}_1\mathbf{Q}\nabla\hat{\phi}_3 \\ \nabla\hat{\phi}_2\mathbf{Q}\nabla\hat{\phi}_1 & \nabla\hat{\phi}_2\mathbf{Q}\nabla\hat{\phi}_2 & \nabla\hat{\phi}_2\mathbf{Q}\nabla\hat{\phi}_3 \\ \nabla\hat{\phi}_3\mathbf{Q}\nabla\hat{\phi}_1 & \nabla\hat{\phi}_3\mathbf{Q}\nabla\hat{\phi}_2 & \nabla\hat{\phi}_3\mathbf{Q}\nabla\hat{\phi}_3 \end{bmatrix} \in \mathbb{R}^{3\times3}, \quad \text{where:}$$

$$\mathbf{Q} = \mathbf{J}_T^{-1}\mathbf{J}_T^{-T} = \begin{bmatrix} (y_3 - y_1)^2 + (x_3 - x_1)^2 & (y_1 - y_2)(y_3 - y_1) + (x_2 - x_1)(x_1 - x_3) \\ (y_1 - y_2)(y_3 - y_1) + (x_2 - x_1)(x_1 - x_3) & (y_2 - y_1)^2 + (x_2 - x_1)^2 \end{bmatrix}$$

Please note that the matrices $\mathbf{Q}$ and $\mathbf{A}^{loc}$ are symmetric. This structural property is highly advantageous for optimizing numerical calculations.

4.3 Pressure Matrices $\mathbf{B}_x$ and $\mathbf{B}_y$ on the P_1-iso-P_1 Macro-Element

Unlike a conforming equal-order pairing, the pressure basis function ψ_j lives on the *macro* triangle $K \in \mathcal{T}_H$, while the velocity basis functions ϕ_i live on the four *child* triangles $T_1, T_2, T_3, T_4 \in \mathcal{T}_h$ obtained by connecting the edge midpoints of K. Consequently the coupling integral must be split into a sum over the four children, and the pressure hat function must be re-expressed in each child's own local reference coordinates. Getting this restriction right is the entire content of the P_1-iso-P_1 assembly.

4.3.1 Macro Geometry and Node Labeling

Let the macro triangle K have vertices X_1, X_2, X_3 with global barycentric (pressure) basis $\psi_1 = \lambda_1,\ \psi_2 = \lambda_2,\ \psi_3 = \lambda_3$. The uniform refinement introduces the three edge midpoints

$$X_4 = \tfrac{1}{2}(X_1 + X_2), \qquad X_5 = \tfrac{1}{2}(X_2 + X_3), \qquad X_6 = \tfrac{1}{2}(X_1 + X_3),$$

giving the four congruent children

$$T_1 = (X_1, X_4, X_6), \quad T_2 = (X_4, X_2, X_5), \quad T_3 = (X_6, X_5, X_3), \quad T_4 = (X_4, X_5, X_6),$$

matching the $\ell_1, \ell_2, \ell_3, \ell_4$ labeling of your Figure 1. Velocity has one degree of freedom at each of the six points $X_1, \ldots, X_6$; pressure has one degree of freedom at each of X_1, X_2, X_3 only.

4.3.2 The Bilinear Form, Split Over Children

$$-\int_K p\,(\nabla\cdot v)\,d\Omega$$

$$-\int_K \psi_j \frac{\partial\phi_i}{\partial x}\,d\Omega = -\sum_{m=1}^{4}\int_{T_m}\psi_j\frac{\partial\phi_i}{\partial x}\,d\Omega = \left|\begin{array}{c}\phi_i\big|_{T_m} \text{ is affine } \Rightarrow \nabla\phi_i \text{ is } \textit{constant} \text{ on } T_m \\ \text{(zero unless } i \text{ is one of } T_m\text{'s three local nodes)}\end{array}\right|$$

$$= -\sum_{m=1}^{4}\left(\frac{\partial\phi_i}{\partial x}\right)_{T_m}\int_{T_m}\psi_j\,d\Omega = -\sum_{m=1}^{4}\left(\frac{\partial\phi_i}{\partial x}\right)_{T_m}|\mathbf{J}_{T_m}|\int_{\hat{T}}\hat{\psi}_j^{(m)}(\xi_1,\xi_2)\,d\xi_1 d\xi_2$$

Everything now hinges on evaluating $\hat{\psi}_j^{(m)}$: the pressure hat function ψ_j, pulled back onto child T_m's *own* reference triangle.

4.3.3 Restricting the Macro Pressure Basis to a Child

Because λ_j is affine on all of K, its restriction to the affine sub-triangle T_m is again affine, and an affine function on $\hat{T}$ is *uniquely* determined by its three nodal values — which is exactly what $\hat{\phi}_1, \hat{\phi}_2, \hat{\phi}_3$ interpolate. Hence:

$$\hat{\psi}_j^{(m)}(\xi_1,\xi_2) \;=\; \lambda_j\left(V_1^{(m)}\right)\hat{\phi}_1(\xi_1,\xi_2) + \lambda_j\left(V_2^{(m)}\right)\hat{\phi}_2(\xi_1,\xi_2) + \lambda_j\left(V_3^{(m)}\right)\hat{\phi}_3(\xi_1,\xi_2)$$

where $V_1^{(m)}, V_2^{(m)}, V_3^{(m)}$ are the three vertices of child T_m (each one of $X_1,\ldots,X_6$), and $\lambda_j(\cdot)$ is evaluated using the known barycentric values

$$\lambda_1 : (X_1,\ldots,X_6)\mapsto \left(1,0,0,\tfrac{1}{2},0,\tfrac{1}{2}\right), \qquad \lambda_2\mapsto\left(0,1,0,\tfrac{1}{2},\tfrac{1}{2},0\right), \qquad \lambda_3\mapsto\left(0,0,1,0,\tfrac{1}{2},\tfrac{1}{2}\right).$$

4.3.4 Exact Integration of the Restricted Pressure Basis

Since $\int_{\hat{T}}\hat{\phi}_k\,d\xi_1 d\xi_2 = \frac{1}{6}$ for $k=1,2,3$ (same identity used for $\mathbf{M}_p^{loc}$ in §5.6), linearity of the integral gives the closed form:

$$w_j^{(m)} \;:=\; \int_{\hat{T}}\hat{\psi}_j^{(m)}(\xi_1,\xi_2)\,d\xi_1 d\xi_2 = \frac{1}{6}\left[\lambda_j\left(V_1^{(m)}\right)+\lambda_j\left(V_2^{(m)}\right)+\lambda_j\left(V_3^{(m)}\right)\right]$$

Evaluating this for all four children and all three pressure nodes yields:

Table 1: Pressure-basis integration weights $w_j^{(m)}$ per child sub-triangle.

Child T_m	$w_1^{(m)}$ (for ψ_1)	$w_2^{(m)}$ (for ψ_2)	$w_3^{(m)}$ (for ψ_3)
$T_1=(X_1,X_4,X_6)$	1/3	1/12	1/12
$T_2=(X_4,X_2,X_5)$	1/12	1/3	1/12
$T_3=(X_6,X_5,X_3)$	1/12	1/12	1/3
$T_4=(X_4,X_5,X_6)$	1/6	1/6	1/6

Note that only the central child T_4 reproduces a uniform $\frac{1}{6}$ across all three pressure nodes — this is the case that (incorrectly) generalizes to the whole macro-element in a naive equal-order assembly. The corner children T_1, T_2, T_3 are systematically weighted *unevenly*: each favors its own corner pressure node by a factor of 4 relative to the other two.

4.3.5 Local Child Matrices $\mathbf{B}_x^{(m)}, \mathbf{B}_y^{(m)}$

Reusing the gradient identity of eq. (2), but built from child T_m's own vertex coordinates $(x_1^{(m)}, y_1^{(m)}), (x_2^{(m)}, y_2^{(m)}), (x_3^{(m)}$ and Jacobian $\mathbf{J}_{T_m}$:

$$\left(\mathbf{B}_x^{(m)}\right)_{ij} = -\left(\frac{\partial \phi_i}{\partial x}\right)_{T_m} |\mathbf{J}_{T_m}|\ w_j^{(m)} = -\Big[(y_2^{(m)} - y_1^{(m)})\tfrac{\partial \hat{\phi}_i}{\partial \xi_2} - (y_3^{(m)} - y_1^{(m)})\tfrac{\partial \hat{\phi}_i}{\partial \xi_1}\Big] w_j^{(m)},$$

$$\left(\mathbf{B}_y^{(m)}\right)_{ij} = -\left(\frac{\partial \phi_i}{\partial y}\right)_{T_m} |\mathbf{J}_{T_m}|\ w_j^{(m)} = \Big[(x_3^{(m)} - x_1^{(m)})\tfrac{\partial \hat{\phi}_i}{\partial \xi_1} - (x_2^{(m)} - x_1^{(m)})\tfrac{\partial \hat{\phi}_i}{\partial \xi_2}\Big] w_j^{(m)},$$

for $i \in \{1,2,3\}$ indexing T_m's three *local* velocity nodes, and $j \in \{1,2,3\}$ indexing K's three macro pressure nodes. Notice the $|\mathbf{J}_{T_m}|$ cancels exactly against the $\frac{1}{|\mathbf{J}_{T_m}|}$ buried inside the gradient — the Jacobian determinant never actually survives into the final formula, exactly as in your original (correct) derivation of $\mathbf{B}^{loc}$ for a single conforming triangle. What changes is *only* $w_j^{(m)}$, replacing your uniform constant $\frac{1}{6}$.

4.3.6 Assembly Into the Macro-Local Block

Each child contributes a 3×3 block coupling its own 3 local velocity nodes to the 3 macro pressure nodes. These are scatter-added into a single 6×3 macro-local matrix indexed by the six fine velocity nodes $(X_1, \ldots, X_6)$:

$$\mathbf{B}_x^K \in \mathbb{R}^{6\times 3}, \qquad \left(\mathbf{B}_x^K\right)_{I(m,i),\,j} \mathrel{+}= \left(\mathbf{B}_x^{(m)}\right)_{ij}, \qquad m = 1, \ldots, 4,$$

where $I(m,i)$ maps T_m's local node i to its global fine-mesh index in $\{1,\ldots,6\}$ (with the three midpoint nodes X_4, X_5, X_6 each shared by exactly two children and therefore receiving two additive contributions). $\mathbf{B}_y^K$ is assembled identically. These macro-local blocks are then assembled over all $K \in \mathcal{T}_H$ into the global $\mathbf{B}_x, \mathbf{B}_y \in \mathbb{R}^{N_p \times N_v}$ exactly as in §5.1.

4.4 Saddle-Point Block Matrix K

By organizing the viscous diffusion blocks and the divergence constraint blocks into a single monolithic operator, we construct the global saddle-point system. This specific structural arrangement takes the classic Karush-Kuhn-Tucker (KKT) form:

$$\begin{bmatrix} \mathbf{A} & 0 & \mathbf{B}_x^T \\ 0 & \mathbf{A} & \mathbf{B}_y^T \\ \mathbf{B}_x & \mathbf{B}_y & 0 \end{bmatrix} \begin{bmatrix} \mathbf{u}_x \\ \mathbf{u}_y \\ \mathbf{p} \end{bmatrix} = \begin{bmatrix} \mathbf{F}_x \\ \mathbf{F}_y \\ \mathbf{0} \end{bmatrix}$$

The block layout reflects the underlying **mixed formulation** (1), where the diagonal velocity stiffness blocks $\mathbf{A}$ remain symmetric positive definite, while the zero block on the lower diagonal highlights the lack of an explicit pressure time-dependence or compressibility stabilizer. The dimensional layout of these individual blocks scales according to the total degrees of freedom allocated to each field variable:

$$\mathbf{A} \in \mathbb{R}^{N_u \times N_u} \quad \mathbf{B}_x \in \mathbb{R}^{N_p \times N_u} \quad \mathbf{B}_x^T \in \mathbb{R}^{N_u \times N_p}$$

4.5 Load Vector F Assembly and Inhomogeneous Boundary Lifting

$$\mathbf{F} = \int_{\Gamma_N} h \cdot v \, d\Gamma - \int_{\Omega} \nu \nabla r_g : \nabla v \, d\Omega = \mathbf{F}_{forces} - \mathbf{A}\vec{r_g} = \begin{bmatrix} F_x \\ F_y \end{bmatrix} - \mathbf{A}\vec{r_g}$$

4.6 Pressure Null-Space Regulation and Singular Constraints

4.7 Sparse Storage Layout and Linear Solver Mechanics

5 Ladyzhenskaya–Babuška–Brezzi Condition

In numerical partial differential equations, the *Ladyzhenskaya–Babuška–Brezzi* (LBB) condition is a sufficient condition for a saddle point problem to have a unique solution that depends continuously on the input data.

At the continuous level, the velocity space V and pressure space Q must satisfy:

$$\exists \beta > 0 : \quad \inf_{q \in Q_{\setminus\{0\}}} \sup_{v \in V_{\setminus\{0\}}} \left(\frac{\int_\Omega q(\nabla \cdot v) d\Omega}{\|v\|_V \cdot \|q\|_Q} \right) \geq \beta$$

5.1 Discrete Mapping and Coordinate Representation

For this problem it is useful to reformulate the condition in a discrete way. We can start by mapping the continuous functions to their nodal coordinate representations:

$$v \to \mathbf{u} = \begin{bmatrix} u_1^x \\ \vdots \\ u_{N_v}^x \\ u_1^y \\ \vdots \\ u_{N_v}^y \end{bmatrix} \in \mathbb{R}^{2N_v}, \quad q \to \mathbf{p} = \begin{bmatrix} p_1 \\ \vdots \\ p_{N_p} \end{bmatrix} \in \mathbb{R}^{N_p}$$

The corresponding discrete differential bilinear forms translate directly into the global block matrices assembled across the mesh:

$$\mathbf{A}_{ij} = \int_\Omega \nu \nabla \phi_i(x,y) \cdot \nabla \phi_j(x,y)\, d\Omega \qquad (\mathbf{B}_x)_{ij} = -\int_\Omega \psi_j(x,y) \frac{\partial \phi_i(x,y)}{\partial x}\, d\Omega \qquad (\mathbf{B}_y)_{ij} = -\int_\Omega \psi_j(x,y) \frac{\partial \phi_i(x,y)}{\partial y}\, d\Omega$$

$$\mathbf{B} = \begin{bmatrix} \mathbf{B}_x & \mathbf{B}_y \end{bmatrix} \qquad \mathbf{M}_v = \begin{bmatrix} \mathbf{A} & \mathbf{0} \\ \mathbf{0} & \mathbf{A} \end{bmatrix} \qquad (\mathbf{M}_p)_{ij} = \int_\Omega \psi_j(x,y)\psi_i(x,y)\, d\Omega$$

Where:

- $\mathbf{u} \in \mathbb{R}^{2N_v}$ is the velocity vector on a fine grid;
- $\mathbf{p} \in \mathbb{R}^{N_p}$ is the pressure vector on a coarse grid;
- $\mathbf{B}_x \in \mathbb{R}^{N_p \times N_v}$ is the discrete horizontal divergence operator;
- $\mathbf{B}_y \in \mathbb{R}^{N_p \times N_v}$ is the discrete vertical divergence operator;
- $\mathbf{B} \in \mathbb{R}^{N_p \times 2N_v}$ is a discrete divergence operator, responsible for coupling pressure and velocity;
- $\mathbf{A} \in \mathbb{R}^{N_v \times N_v}$ is a discrete Laplacian operator that handles the viscosity of the fluid;
- $\mathbf{M}_v \in \mathbb{R}^{2N_v \times 2N_v}$ is the velocity energy weight matrix, used to calculate the physical size and kinetic energy of the velocity field.
- $\mathbf{M}_p \in \mathbb{R}^{N_p \times N_p}$ is the pressure mass matrix that computes the spatial overlap of pressure basis functions.

5.2 Discretization Proofs for Operators and Field Norms

Using the coordinate definitions above, we can systematically transform the three core components of the continuous inf-sup condition into algebraic matrix operations:

1. $$\int_\Omega q(\nabla\cdot v)\,d\Omega = \left| \begin{array}{ccc} q(x,y)\approx\sum_{j=1}^{N_p} p_j\psi_j(x,y) & v_x(x,y)\approx\sum_{i=1}^{N_v} u_i^x\phi_i(x,y) & \frac{\partial v_x}{\partial x}\approx\sum_{i=1}^{N_v} u_i^x\frac{\partial}{\partial x}\phi_i(x,y) \\ v(x,y)=\begin{bmatrix} v_x(x,y)\\ v_y(x,y)\end{bmatrix} & v_y(x,y)\approx\sum_{i=1}^{N_v} u_i^y\phi_i(x,y) & \frac{\partial v_y}{\partial y}\approx\sum_{i=1}^{N_v} u_i^y\frac{\partial}{\partial y}\phi_i(x,y) \end{array}\right| =$$

$$= \int_\Omega q(x,y)\left(\begin{bmatrix}\frac{\partial}{\partial x}\\ \frac{\partial}{\partial y}\end{bmatrix}^T \begin{bmatrix} v_x(x,y)\\ v_y(x,y)\end{bmatrix}\right)d\Omega = \int_\Omega q(x,y)\left(\frac{\partial v_x}{\partial x}+\frac{\partial v_y}{\partial y}\right)d\Omega =$$

$$= \int_\Omega q(x,y)\frac{\partial v_x}{\partial x}\,d\Omega + \int_\Omega q(x,y)\frac{\partial v_y}{\partial y}\,d\Omega =$$

$$= \int_\Omega \sum_{j=1}^{N_p} p_j\psi_j(x,y)\sum_{i=1}^{N_v} u_i^x\frac{\partial}{\partial x}\phi_i(x,y)\,d\Omega + \int_\Omega \sum_{j=1}^{N_p} p_j\psi_j(x,y)\sum_{i=1}^{N_v} u_i^y\frac{\partial}{\partial y}\phi_i(x,y)\,d\Omega =$$

$$= \left[\sum_{j=1}^{N_p}\sum_{i=1}^{N_v} p_j\cdot\int_\Omega \psi_j(x,y)\frac{\partial}{\partial x}\phi_i(x,y)\,d\Omega\cdot u_i^x\right] + \left[\sum_{j=1}^{N_p}\sum_{i=1}^{N_v} p_j\cdot\int_\Omega \psi_j(x,y)\frac{\partial}{\partial y}\phi_i(x,y)\,d\Omega\cdot u_i^y\right]$$

$$= \sum_{j=1}^{N_p}\sum_{i=1}^{N_v} p_j\,(B_x)_{ji}\,u_i^x + \sum_{j=1}^{N_p}\sum_{i=1}^{N_v} p_j\,(B_y)_{ji}\,u_i^y = \mathbf{p}^T\mathbf{B}_x\mathbf{u}_x + \mathbf{p}^T\mathbf{B}_y\mathbf{u}_y = \mathbf{p}^T\begin{bmatrix}\mathbf{B}_x & \mathbf{B}_y\end{bmatrix}\begin{bmatrix}\mathbf{u}_x\\ \mathbf{u}_y\end{bmatrix} =$$

$$= \mathbf{p}^T\mathbf{B}\mathbf{u}$$

2. $$\|v\|_V = \left(\int_\Omega \left(\|\nabla v_x\|^2+\|\nabla v_y\|^2\right)\,d\Omega\right)^{1/2} = \left(\int_\Omega \left(\nabla v_x\cdot\nabla v_x + \nabla v_y\cdot\nabla v_y\right)\,d\Omega\right)^{1/2} =$$

$$= \left| \begin{array}{cc} v_x(x,y)\approx\sum_{i=1}^{N_v} u_i^x\phi_i(x,y) & \nabla v_x(x,y)\approx\sum_{i=1}^{N_v} u_i^x\nabla\phi_i(x,y) \\ v_y(x,y)\approx\sum_{j=1}^{N_v} u_j^y\phi_j(x,y) & \nabla v_y(x,y)\approx\sum_{j=1}^{N_v} u_j^y\nabla\phi_j(x,y) \end{array}\right| =$$

$$= \left(\int_\Omega\left(\sum_{i=1}^{N_v} u_i^x\nabla\phi_i(x,y)\cdot\sum_{j=1}^{N_v} u_j^x\nabla\phi_j(x,y) + \sum_{i=1}^{N_v} u_i^y\nabla\phi_i(x,y)\cdot\sum_{j=1}^{N_v} u_j^y\nabla\phi_j(x,y)\right)\,d\Omega\right)^{1/2} =$$

$$= \left(\left[\sum_{i=1}^{N_v}\sum_{j=1}^{N_v} u_i^x\cdot\int_\Omega \nabla\phi_i(x,y)\cdot\nabla\phi_j(x,y)\,d\Omega\cdot u_j^x\right] + \sum_{i=1}^{N_v}\sum_{j=1}^{N_v}\left[u_i^y\cdot\int_\Omega \nabla\phi_i(x,y)\cdot\nabla\phi_j(x,y)\,d\Omega\cdot u_j^y\right]\right)^{1/2} =$$

$$= \left(\left[\sum_{i=1}^{N_v}\sum_{j=1}^{N_v} u_i^x A_{ij} u_j^x\right] + \left[\sum_{i=1}^{N_v}\sum_{j=1}^{N_v} u_i^y A_{ij} u_j^y\right]\right)^{1/2} =$$

$$= \left(\mathbf{u_x}^T\mathbf{A}\mathbf{u_x} + \mathbf{u_y}^T\mathbf{A}\mathbf{u_y}\right)^{1/2} = \left(\begin{bmatrix}\mathbf{u_x}\\ \mathbf{u_y}\end{bmatrix}^T\begin{bmatrix}\mathbf{A} & \mathbf{0}\\ \mathbf{0} & \mathbf{A}\end{bmatrix}\begin{bmatrix}\mathbf{u_x}\\ \mathbf{u_y}\end{bmatrix}\right)^{1/2} = \left(\mathbf{u}^T\mathbf{M}_v\mathbf{u}\right)^{1/2}$$

3. $$\|q\|_Q = \left(\int_\Omega q(x,y)^2\,d\Omega\right)^{1/2} = \left| q(x,y)\approx\sum_{i=1}^{N_p} p_i\psi_i(x,y)\right| =$$

$$= \left(\int_\Omega \sum_{i=1}^{N_p} p_i\psi_i(x,y)\cdot\sum_{j=1}^{N_p} p_j\psi_j(x,y)\,d\Omega\right)^{1/2} = \left(\int_\Omega \sum_{i=1}^{N_p}\sum_{j=1}^{N_p} p_i p_j\psi_j(x,y)\psi_i(x,y)\,d\Omega\right)^{1/2} =$$

$$= \left(\sum_{i=1}^{N_p}\sum_{j=1}^{N_p} p_i\cdot\left(\int_\Omega \psi_j(x,y)\psi_i(x,y)\,d\Omega\right)\cdot p_j\right)^{1/2} = \left(\sum_{i=1}^{N_p}\sum_{j=1}^{N_p} p_i\cdot(M_p)_{ij}\cdot p_j\right)^{1/2} = \left(\mathbf{p}^T\mathbf{M}_p\mathbf{p}\right)^{1/2}$$

5.3 The Final Discrete Matrix Form

By substituting our three derived algebraic expressions directly back into the continuous supremum framework, the continuous inf-sup condition simplifies into its final, discrete matrix form:

$$\exists \beta > 0 : \quad \inf_{\mathbf{p}\neq\mathbf{0}} \sup_{\mathbf{u}\neq\mathbf{0}} \left(\frac{\mathbf{p}^T\mathbf{B}\mathbf{u}}{(\mathbf{u}^T\mathbf{M}_v\mathbf{u})^{1/2} \cdot (\mathbf{p}^T\mathbf{M}_p\mathbf{p})^{1/2}} \right) \geq \beta$$

This discrete fraction acts as a normalized compatibility test, using $\mathbf{M}_v$ and $\mathbf{M}_p$ to ensure that the fields are measured by their true physical sizes rather than arbitrary numbers. Because calculating a nested maximum and minimum over thousands of mesh nodes is impossible for a computer, we must use a mathematical shortcut to transform this fraction into a simple matrix eigenvalue problem.

5.4 Solving the Optimization Problem

The optimization problem is of the following appearence:

$$\min_{\mathbf{p}\neq\mathbf{0}} \max_{\mathbf{u}\neq\mathbf{0}} \left(\frac{\mathbf{p}^T\mathbf{B}\mathbf{u}}{(\mathbf{u}^T\mathbf{M}_v\mathbf{u})^{1/2} \cdot (\mathbf{p}^T\mathbf{M}_p\mathbf{p})^{1/2}} \right)$$

We start with the maximization of with respect to $\mathbf{u}$:

$$\max_{\mathbf{u}\neq\mathbf{0}} \left(\frac{\mathbf{p}^T\mathbf{B}\mathbf{u}}{(\mathbf{u}^T\mathbf{M}_v\mathbf{u})^{\frac{1}{2}} \cdot (\mathbf{p}^T\mathbf{M}_p\mathbf{p})^{\frac{1}{2}}} \right) = \frac{1}{(\mathbf{p}^T\mathbf{M}_p\mathbf{p})^{\frac{1}{2}}} \max_{\mathbf{u}\neq\mathbf{0}} \left(\frac{\mathbf{p}^T\mathbf{B}\mathbf{u}}{(\mathbf{u}^T\mathbf{M}_v\mathbf{u})^{\frac{1}{2}}} \right) = \left| \begin{array}{c} \mathbf{M}_v = \mathbf{M}_v^{\frac{1}{2}}\mathbf{M}_v^{\frac{1}{2}} \\ \mathbf{w} = \mathbf{M}_v^{\frac{1}{2}}\mathbf{u}, \quad \mathbf{u} = \mathbf{M}_v^{-\frac{1}{2}}\mathbf{w} \end{array} \right| =$$

$$= \frac{1}{(\mathbf{p}^T\mathbf{M}_p\mathbf{p})^{\frac{1}{2}}} \max_{\mathbf{w}\neq\mathbf{0}} \left(\frac{\mathbf{p}^T\mathbf{B}\left(\mathbf{M}_v^{-\frac{1}{2}}\mathbf{w}\right)}{\left(\left(\mathbf{M}_v^{-\frac{1}{2}}\mathbf{w}\right)^T \mathbf{M}_v \left(\mathbf{M}_v^{-\frac{1}{2}}\mathbf{w}\right)\right)^{\frac{1}{2}}} \right) =$$

$$= \frac{1}{(\mathbf{p}^T\mathbf{M}_p\mathbf{p})^{\frac{1}{2}}} \max_{\mathbf{w}\neq\mathbf{0}} \left(\frac{\mathbf{p}^T\mathbf{B}\mathbf{M}_v^{-\frac{1}{2}}\mathbf{w}}{\left(\mathbf{w}^T\mathbf{M}_v^{-\frac{1}{2}}\mathbf{M}_v^{\frac{1}{2}}\mathbf{M}_v^{\frac{1}{2}}\mathbf{M}_v^{-\frac{1}{2}}\mathbf{w}\right)^{\frac{1}{2}}} \right) =$$

$$= \frac{1}{(\mathbf{p}^T\mathbf{M}_p\mathbf{p})^{\frac{1}{2}}} \max_{\mathbf{w}\neq\mathbf{0}} \frac{\left(\mathbf{M}_v^{-\frac{1}{2}}\mathbf{B}^T\mathbf{p}\right)^T \mathbf{w}}{\|\mathbf{w}\|_2} = \left| \begin{array}{c} \mathbf{a} = c\mathbf{x} \\ \max_{\mathbf{a}\neq\mathbf{0}} \frac{\mathbf{x}^T\mathbf{a}}{\|\mathbf{a}\|_2} = \max_{\mathbf{x}\neq\mathbf{0}} \frac{c\mathbf{x}^T\mathbf{x}}{c\|\mathbf{x}\|_2} = \|\mathbf{x}\|_2 \end{array} \right| =$$

$$= \frac{1}{(\mathbf{p}^T\mathbf{M}_p\mathbf{p})^{\frac{1}{2}}} \|\mathbf{M}_v^{-\frac{1}{2}}\mathbf{B}^T\mathbf{p}\|_2$$

With the maximization out of the way it is now possible to minimize over pressure $\mathbf{p}$:

$$\min_{\mathbf{p}\neq\mathbf{0}} \frac{\|\mathbf{M}_v^{-\frac{1}{2}}\mathbf{B}^T\mathbf{p}\|_2}{(\mathbf{p}^T\mathbf{M}_p\mathbf{p})^{\frac{1}{2}}} \quad \longrightarrow \quad \left(\min_{\mathbf{p}\neq\mathbf{0}} \frac{\|\mathbf{M}_v^{-\frac{1}{2}}\mathbf{B}^T\mathbf{p}\|_2}{(\mathbf{p}^T\mathbf{M}_p\mathbf{p})^{\frac{1}{2}}} \right)^2 = \min_{\mathbf{p}\neq\mathbf{0}} \frac{\left(\mathbf{M}_v^{-\frac{1}{2}}\mathbf{B}^T\mathbf{p}\right)^T \left(\mathbf{M}_v^{-\frac{1}{2}}\mathbf{B}^T\mathbf{p}\right)}{\mathbf{p}^T\mathbf{M}_p\mathbf{p}} =$$

$$= \min_{\mathbf{p}\neq\mathbf{0}} \frac{\mathbf{p}^T\mathbf{B}\mathbf{M}_v^{-\frac{1}{2}}\mathbf{M}_v^{-\frac{1}{2}}\mathbf{B}^T\mathbf{p}}{\mathbf{p}^T\mathbf{M}_p\mathbf{p}} = \min_{\mathbf{p}\neq\mathbf{0}} \frac{\mathbf{p}^T\left(\mathbf{B}\mathbf{M}_v^{-1}\mathbf{B}^T\right)\mathbf{p}}{\mathbf{p}^T\mathbf{M}_p\mathbf{p}} = \left| \begin{array}{c} R(\mathbf{A},\mathbf{B};\mathbf{x}) = \frac{\mathbf{x}^T\mathbf{A}\mathbf{x}}{\mathbf{x}^T\mathbf{B}\mathbf{x}} \\ \mathbf{q} = \mathbf{M}_p^{1/2}\mathbf{p} \\ \mathbf{p} = \mathbf{M}_p^{-1/2}\mathbf{q} \end{array} \right| =$$

$$= \min_{\mathbf{q}\neq\mathbf{0}} \frac{\left(\mathbf{M}_p^{-1/2}\mathbf{q}\right)^T \left(\mathbf{B}\mathbf{M}_v^{-1}\mathbf{B}^T\right)\left(\mathbf{M}_p^{-1/2}\mathbf{q}\right)}{\left(\mathbf{M}_p^{-1/2}\mathbf{q}\right)^T \mathbf{M}_p \left(\mathbf{M}_p^{-1/2}\mathbf{q}\right)} = \min_{\mathbf{q}\neq\mathbf{0}} \frac{\mathbf{q}^T\left(\mathbf{M}_p^{-1/2}\mathbf{B}\mathbf{M}_v^{-1}\mathbf{B}^T\mathbf{M}_p^{-1/2}\right)\mathbf{q}}{\mathbf{q}^T\mathbf{q}} =$$

$$= \left| \begin{array}{c} \text{Rayleigh quotient:} \\ R(\mathbf{K};\mathbf{x}) = \frac{\mathbf{x}^T\mathbf{K}\mathbf{x}}{\mathbf{x}^T\mathbf{x}} \\ \mathbf{K} = \mathbf{M}_p^{-1/2}\mathbf{B}\mathbf{M}_v^{-1}\mathbf{B}^T\mathbf{M}_p^{-1/2} \end{array} \right| = \min_{\mathbf{q}\neq\mathbf{0}} R(\mathbf{K};\mathbf{q}) = \lambda_{min}(\mathbf{K})$$

To compute minimum of the Rayleigh quotient we would need to compute its minimal eigenvalue and get rid of the substitution $\mathbf{q}$:

$$\mathbf{K}\mathbf{q} = \lambda\mathbf{q}$$

$$\begin{aligned} \mathbf{K}\mathbf{M}_p^{1/2}\mathbf{p} &= \lambda\mathbf{M}_p^{1/2}\mathbf{p} \quad \Big| \cdot \mathbf{M}_p^{-1/2} \\ \mathbf{M}_p^{-1/2}\mathbf{K}\mathbf{M}_p^{1/2}\mathbf{p} &= \lambda\mathbf{M}_p^{-1/2}\mathbf{M}_p^{1/2}\mathbf{p} \\ \mathbf{M}_p^{-1/2}\left(\mathbf{M}_p^{-1/2}\mathbf{B}\mathbf{M}_v^{-1}\mathbf{B}^T\mathbf{M}_p^{-1/2}\right)\mathbf{M}_p^{1/2}\mathbf{p} &= \lambda\mathbf{p} \\ \left(\mathbf{M}_p^{-1}\mathbf{B}\mathbf{M}_v^{-1}\mathbf{B}^T\right)\mathbf{p} &= \lambda\mathbf{p} \end{aligned}$$

$$\lambda_{min}\left(\hat{\mathbf{K}}\right), \quad \text{where: } \hat{\mathbf{K}} = \mathbf{M}_p^{-1}\mathbf{B}\mathbf{M}_v^{-1}\mathbf{B}^T$$

We also have to revert the squaring of the term used earlier:

$$\left(\min_{\mathbf{p}\neq 0} \frac{\|\mathbf{M}_v^{-\frac{1}{2}}\mathbf{B}^T\mathbf{p}\|_2}{(\mathbf{p}^T\mathbf{M}_p\mathbf{p})^{\frac{1}{2}}}\right)^2 = \lambda_{min}\left(\hat{\mathbf{K}}\right)$$

$$\min_{\mathbf{p}\neq\mathbf{0}}\max_{\mathbf{u}\neq\mathbf{0}}\left(\frac{\mathbf{p}^T\mathbf{B}\mathbf{u}}{(\mathbf{u}^T\mathbf{M}_v\mathbf{u})^{1/2}\cdot(\mathbf{p}^T\mathbf{M}_p\mathbf{p})^{1/2}}\right) = \sqrt{\lambda_{min}\left(\hat{\mathbf{K}}\right)}$$

5.5 Numerical Stability Assessment

Lastly, we can look at the stability condition in its final, practical form. In a two-dimensional simulation, the velocity field is split into horizontal and vertical components. Because of this, the global divergence matrix $\mathbf{B}$ and the velocity matrix $\mathbf{M}_v$ must be expanded into their individual block components $\mathbf{B}_x$, $\mathbf{B}_y$, and the velocity stiffness matrix $\mathbf{A}$.

Substituting these block matrices into our eigenvalue problem yields the fully expanded discrete inf-sup condition:

$$\exists\beta > 0: \quad \inf_{\mathbf{p}\neq\mathbf{0}}\sup_{\mathbf{u}\neq\mathbf{0}}\left(\frac{\mathbf{p}^T\mathbf{B}\mathbf{u}}{(\mathbf{u}^T\mathbf{M}_v\mathbf{u})^{1/2}\cdot(\mathbf{p}^T\mathbf{M}_p\mathbf{p})^{1/2}}\right) = \sqrt{\lambda_{min}\left(\hat{\mathbf{K}}\right)} \geq \beta$$

$$\exists\beta > 0: \quad \sqrt{\lambda_{min}\left(\mathbf{M}_p^{-1}\mathbf{B}\mathbf{M}_v^{-1}\mathbf{B}^T\right)} = \sqrt{\lambda_{min}\left(\mathbf{M}_p^{-1}\begin{bmatrix}\mathbf{B}_x & \mathbf{B}_y\end{bmatrix}\begin{bmatrix}\mathbf{A} & \mathbf{0}\\ \mathbf{0} & \mathbf{A}\end{bmatrix}^{-1}\begin{bmatrix}\mathbf{B}_x^T\\ \mathbf{B}_y^T\end{bmatrix}\right)} \geq \beta$$

$$\exists\beta > 0: \quad \sqrt{\lambda_{min}\left(\mathbf{M}_p^{-1}\left(\mathbf{B}_x\mathbf{A}^{-1}\mathbf{B}_x^T + \mathbf{B}_y\mathbf{A}^{-1}\mathbf{B}_y^T\right)\right)} \geq \beta$$

The minimum eigenvalue λ_{min} serves as a direct numerical test for the stability of our chosen velocity and pressure finite element grids. To verify if a grid combination is safe to use, we look at how λ_{min} behaves as the mesh is refined:

- **Stable Elements:** If the chosen grid pair is stable, λ_{min} will level off and remain strictly greater than a positive constant ($\lambda_{\text{min}} \geq \beta^2 > 0$) even on very fine meshes. This guarantees a clean, reliable solution.
- **Unstable Elements:** If the grid pair is unstable, λ_{min} will continuously drop toward zero as the mesh size decreases. This acts as a clear warning that the simulation will suffer from fake pressure spikes and oscillations (often called checkerboard patterns).

5.6 Mass Matrix $\mathbf{M}_p$

To check our results numerically we will have to construct the pressure mass matrix as well:

$$(\mathbf{M}_p)_{ij} = \int_\Omega \psi_j(x,y)\psi_i(x,y)\,d\Omega = \sum_T \int_T \psi_j(x,y)\psi_i(x,y)dxdy = \sum_T \int_{\hat{T}} \hat{\psi}_j(\xi_1,\xi_2)\hat{\psi}_i(\xi_1,\xi_2)\,|\mathbf{J}_T|\;d\xi_1 d\xi_2$$

$$= \sum_T |\mathbf{J}_T| \int_{\hat{T}} \hat{\psi}_j\hat{\psi}_i\,d\xi_1 d\xi_2$$

We can now turn to the local mass matrix:

$$\mathbf{M}_p^{loc} = |\mathbf{J}_T| \int_{\hat{T}} \hat{\psi}_j\hat{\psi}_i\,d\xi_1 d\xi_2 = |\mathbf{J}_T| \int_{\hat{T}} \begin{bmatrix} \hat{\psi}_1\hat{\psi}_1 & \hat{\psi}_1\hat{\psi}_2 & \hat{\psi}_1\hat{\psi}_3 \\ \hat{\psi}_2\hat{\psi}_1 & \hat{\psi}_2\hat{\psi}_2 & \hat{\psi}_2\hat{\psi}_3 \\ \hat{\psi}_3\hat{\psi}_1 & \hat{\psi}_3\hat{\psi}_2 & \hat{\psi}_3\hat{\psi}_3 \end{bmatrix} d\xi_1 d\xi_2 = \left| \begin{array}{l} \hat{\psi}_1 = 1-\xi_1-\xi_2 \\ \hat{\psi}_2 = \xi_1 \\ \hat{\psi}_3 = \xi_2 \end{array} \right| =$$

$$= |\mathbf{J}_T| \int_0^1 \int_0^{1-\xi_1} \begin{bmatrix} (1-\xi_1-\xi_2)(1-\xi_1-\xi_2) & \xi_1(1-\xi_1-\xi_2) & \xi_2(1-\xi_1-\xi_2) \\ \xi_1(1-\xi_1-\xi_2) & \xi_1\xi_1 & \xi_2\xi_1 \\ \xi_2(1-\xi_1-\xi_2) & \xi_2\xi_1 & \xi_2\xi_2 \end{bmatrix} d\xi_2 d\xi_1 =$$

$$= \frac{|\mathbf{J}_T|}{24} \begin{bmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{bmatrix}$$

6 Reverse Techniques: Einsum

6.1 First Einsum

```
A_local = np.zeros(shape=(Nt, 3, 3))
for i in range(3):
    for j in range(i, 3):

        grad_i = test_function_derivatives[i]
        grad_j = test_function_derivatives[j]

        val = np.einsum('i,nij,j->n',
                        grad_j, Q_mat, grad_i)

        val *= kinematic_viscosity / (2 * det_J)

        A_local[:, i, j] = val
        A_local[:, j, i] = val
```

$$\underbrace{[\psi_1, \psi_2]}_{\psi_i} \quad \underbrace{\begin{bmatrix} q_{11}^n & q_{12}^n \\ q_{21}^n & q_{22}^n \end{bmatrix}}_{q_{ij}^n} \quad \underbrace{[\psi_1, \psi_2]}_{\psi_j}$$

$$\sum_{i=2}^{2} \sum_{j=2}^{2} \psi_i q_{ij}^n \psi_j = A^n \in \mathbb{R}^n$$

6.2 Second Einsum

```
A_local = np.einsum('mi,txy,nj->tmn',
                    test_function_derivatives, Q_mat, test_function_derivatives)
A_local *= (kinematic_viscosity / (2.0 * det_J))[:, None, None]
```

$$\underbrace{\begin{bmatrix} [\psi_1^1, \psi_2^1] \\ [\psi_1^2, \psi_2^2] \\ [\psi_1^3, \psi_2^3] \end{bmatrix}}_{\psi_i^m} \quad \underbrace{\begin{bmatrix} q_{11}^t & q_{12}^t \\ q_{21}^t & q_{22}^t \end{bmatrix}}_{q_{xy}^t} \quad \underbrace{\begin{bmatrix} [\psi_1^1, \psi_2^1] \\ [\psi_1^2, \psi_2^2] \\ [\psi_1^3, \psi_2^3] \end{bmatrix}}_{\psi_j^n} \longrightarrow \underbrace{\begin{bmatrix} A_{11}^t & A_{12}^t & A_{13}^t \\ A_{21}^t & A_{22}^t & A_{23}^t \\ A_{31}^t & A_{32}^t & A_{33}^t \end{bmatrix}}_{A_{mn}^t}$$

So by using indexation and sums we notice that the structure has not really changed since the first example 6.1:

$$\sum_{i=1}^{2} \sum_{j=1}^{2} \psi_i^m q_{ij}^t \psi_j^n = A_{mn}^t \in \mathbb{R}^{t \times m \times n}$$

7 Numerical Experiments and Applications

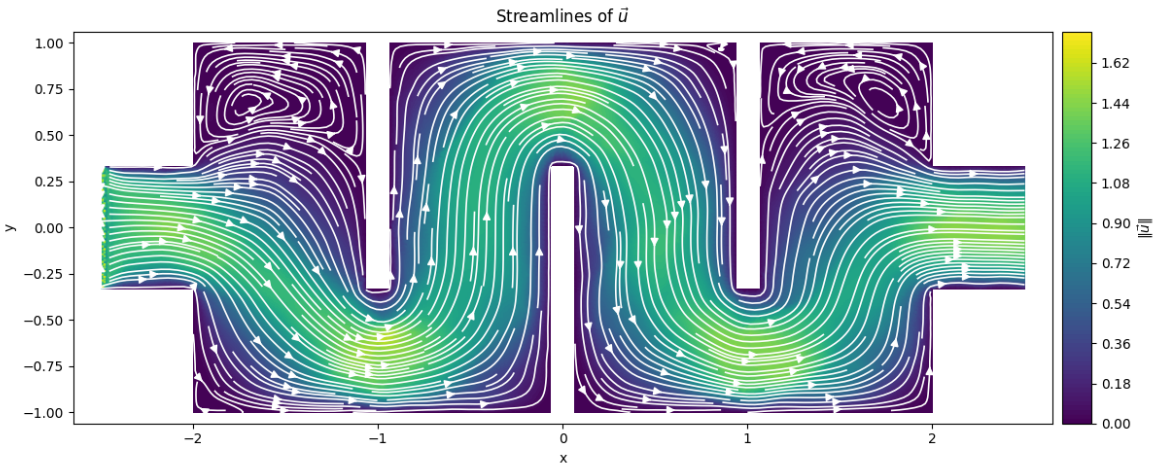
In this section, we validate our P_1-iso-P_1 finite element solver using two distinct fluid engineering case studies: an *Exchanger Device* and a *Winding Pipe*. Both configurations operate under a creeping flow regime (Re $\ll 1$), where viscous forces dominate.

For each case study, the numerical results are presented in a vertical sequence showing:

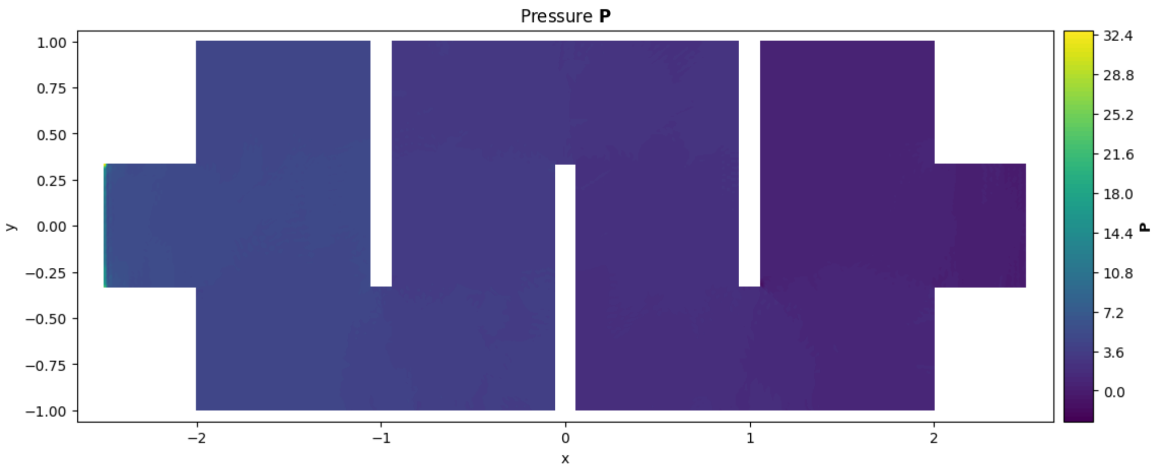
1. **Pressure Field Contour Maps:** Tri-contour plots displaying the continuous linear scalar pressure fields ($\mathbf{P}$) and revealing the pressure drop across the domains.

2. **Velocity Field Streamplots:** Streamline trajectories displaying the path lines, vector flow patterns, and boundary layer behaviors of the velocity field ($\mathbf{u}$).

7.1 Exchanger Device



(a) Computed Velocity Streamline Vector Paths.



(b) Continuous Piecewise Linear Pressure Contour Map.

Figure 1: Complete Stokes flow simulation inside the *Exchanger Device* domain.

The numerical results inside the *Exchanger Device* reveal distinct kinematic adjustments driven by the variable cross-sectional area of the channel network. As observed in the velocity streamline distributions (Figure 1a), the fluid velocity increases significantly at the turns, where the geometry narrows relative to the wider vertical passages. To preserve mass conservation under the incompressibility constraint $\nabla \cdot \mathbf{u} = 0$, the velocity field must accelerate through these local contractions, creating higher localized velocity gradients along the curved inner walls.

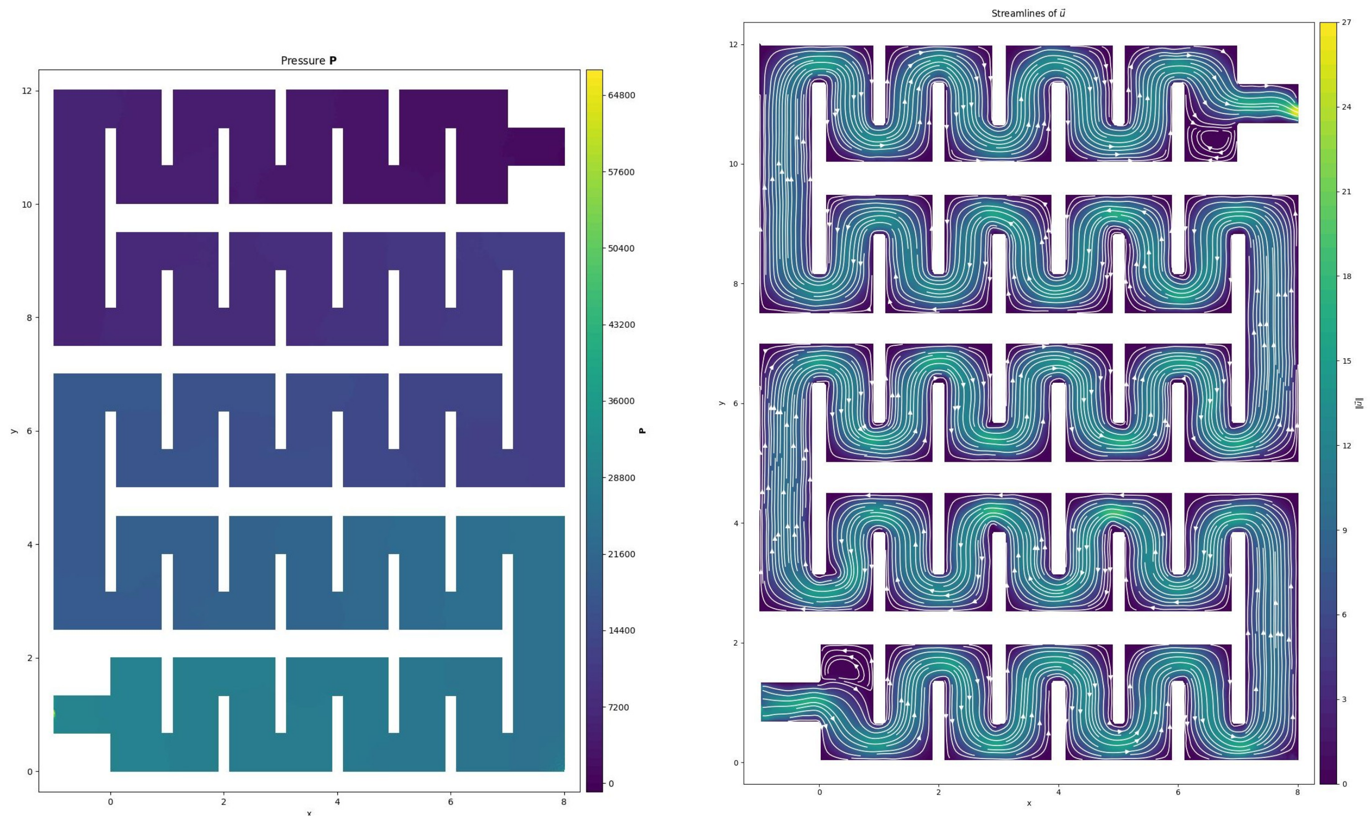
Furthermore, the simulation clearly shows fluid swirls (recirculation zones) inside the dead-end, cut-off extensions. Because Stokes flow has no inertia, the fluid balance simplifies to a strict equilibrium between pressure gradients and viscous friction:

$$-\nu \Delta \mathbf{u} + \nabla p = \mathbf{0}$$

As a result, the main stream takes the path of least resistance through the primary central channels. However, as this fast-moving core flow sweeps past the stagnant fluid trapped in the side cavities, it shears against it. This boundary shear transfers momentum across the open gap, dragging the passive fluid along and creating slow, circular vortices inside the extensions. The computed velocity field exhibits well-resolved stagnation regions and sharp shear interfaces, indicating stable and physically consistent behavior of the mixed finite element discretization.

7.2 Winding Pipe

The *Winding Pipe* models an internal distribution manifold containing geometric obstacles designed to alter flow paths. This setup presents a challenging test case for mass conservation preservation across sudden transitions.



(a) Continuous Piecewise Linear Pressure Contour Map.

(b) Computed Velocity Streamline Vector Paths.

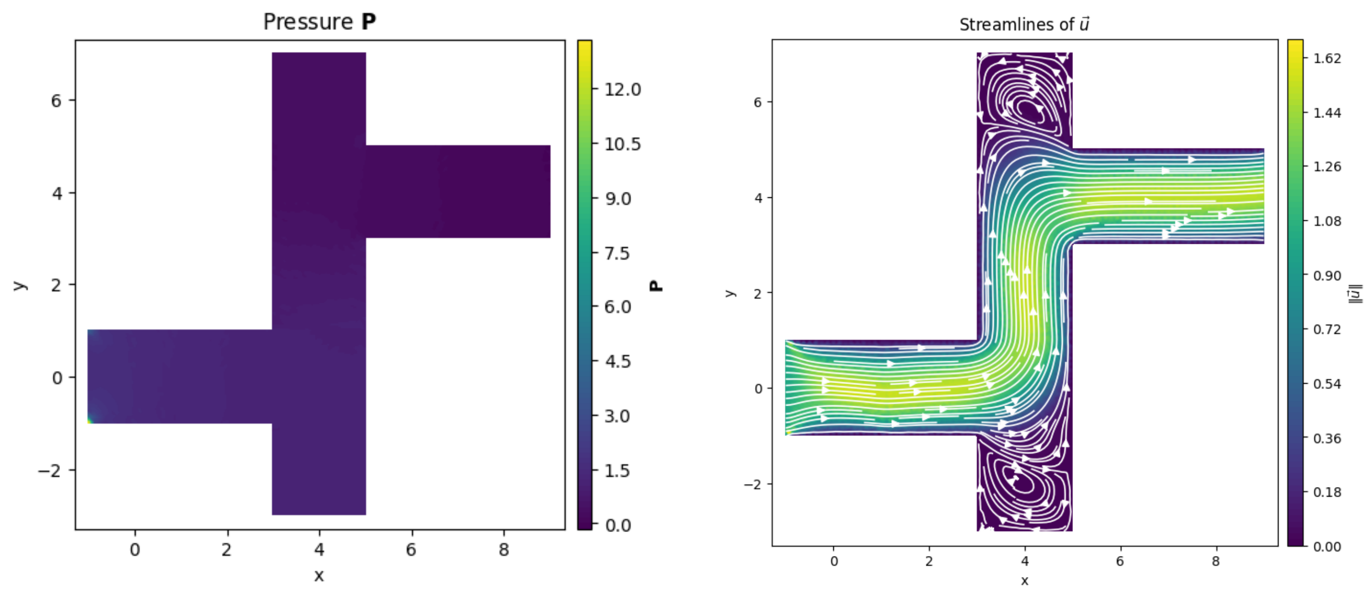
Figure 2: Complete Stokes flow simulation inside the *Winding Pipe* domain.

As observed in Figure 3, the velocity streamlines adapt smoothly around the internal channel obstacles without displaying spurious velocity vector spikes. Concurrently, the pressure distribution field (Figure 2a) displays a continuous linear gradient drop, confirming the inf-sup spatial stability of the dual-grid discretization matrix configuration.

8 Convergence of Numerical Solutions

8.1 Embedding Matrix

8.2 Convergence Analysis via a Reference Solution on a Z-Shaped Channel Device



(a) Continuous Piecewise Linear Pressure Contour Map. (b) Computed Velocity Streamline Vector Paths.

Figure 3: Complete Stokes flow simulation inside the *Z-Shaped Channel Device* domain.

To investigate the convergence behavior of the P1-iso-P1 discretization, a reference solution was first computed on the finest available mesh, obtained after seven uniform refinements of the initial domain geometry. Solutions on coarser meshes were then compared against this reference solution.

Since the discrete velocity and pressure spaces have different dimensions on different refinement levels, the solutions cannot be compared directly. Therefore, each coarse-grid solution was recursively transferred to the reference mesh using the finite-element embedding operator returned by `embeddingForRefined`. For a solution computed on refinement level i, the embedding matrices corresponding to all subsequent refinements were applied successively,

$$\mathbf{u}_{\text{emb}}^{(i)} = \mathbf{E}_N \cdots \mathbf{E}_{i+1}\mathbf{E}_i\mathbf{u}^{(i)},$$

and analogously for the pressure field. After each embedding step, the mesh was refined and the next embedding matrix was constructed on the newly generated mesh. This recursive procedure produced velocity and pressure approximations living in the same finite-element spaces as the reference solution.

The errors were then defined by

$$\mathbf{e}_{u_x} = \mathbf{u}_{x,\text{emb}} - \mathbf{u}_{x,\text{ref}}, \qquad \mathbf{e}_{u_y} = \mathbf{u}_{y,\text{emb}} - \mathbf{u}_{y,\text{ref}},$$

and

$$\mathbf{e}_p = \mathbf{p}_{\text{emb}} - \mathbf{p}_{\text{ref}}.$$

Their discrete L^2-norms were evaluated using the mass matrices corresponding to the fine and coarse spaces on the reference mesh,

$$\|\mathbf{e}_u\|_{L^2(\Omega)} = \sqrt{\mathbf{e}_{u_x}^T \mathbf{M}_{\text{fine}} \mathbf{e}_{u_x} + \mathbf{e}_{u_y}^T \mathbf{M}_{\text{fine}} \mathbf{e}_{u_y}},$$

and

$$\|\mathbf{e}_p\|_{L^2(\Omega)} = \sqrt{\mathbf{e}_p^T \mathbf{M}_{\text{coarse}} \mathbf{e}_p}.$$

Finally, the experimental convergence rates were estimated from two consecutive refinement levels according to

$$r = \frac{\log(E_h/E_{h/2})}{\log(2)},$$

where E_h denotes the corresponding L^2-error.

Table 2: Stokes FEM Error Convergence Analysis on the Z-Shaped Channel Geometric Mesh.

Level	**Mesh Size**	**Velocity DOFs**	**Pressure DOFs**	L^2 **Error u**	**Rate** (r_u)	L^2 **Error** p	**Rate** (r_p)
0	$h_0/1$	114	20	39.5400	—	4826.3	—
1	$h_0/2$	370	57	27.2150	0.539	4804.2	0.007
2	$h_0/4$	1314	185	11.9470	1.188	4725.1	0.024
3	$h_0/8$	4930	657	4.9725	1.265	4571.5	0.048
4	$h_0/16$	19074	2465	2.3325	1.092	4266.3	0.100
5	$h_0/32$	75010	9537	1.2049	0.953	3657.0	0.222
6	$h_0/64$	297474	37505	0.7305	0.722	2438.1	0.585

Table 2 demonstrates a consistent reduction of both the velocity and pressure errors as the mesh is refined, indicating convergence of the numerical solution toward the reference solution. The velocity error decreases from 39.5400 on the coarsest mesh to 0.7305 on the finest mesh considered, with observed convergence rates oscillating between 0.54 and 1.27. The pressure error also decreases, although at a significantly slower rate, from 4826.3 to 2438.1. This behavior is expected, as the pressure approximation is computed on the coarser mesh of the P1-iso-P1 discretization and therefore converges more slowly than the velocity field. Additionally, the re-entrant corners inherent to this specific device geometry introduce flow singularities that naturally degrade the theoretical asymptotic convergence rates. Overall, the results confirm that mesh refinement improves the accuracy of both solution components and provide numerical evidence for the convergence of the implemented Stokes finite element method.

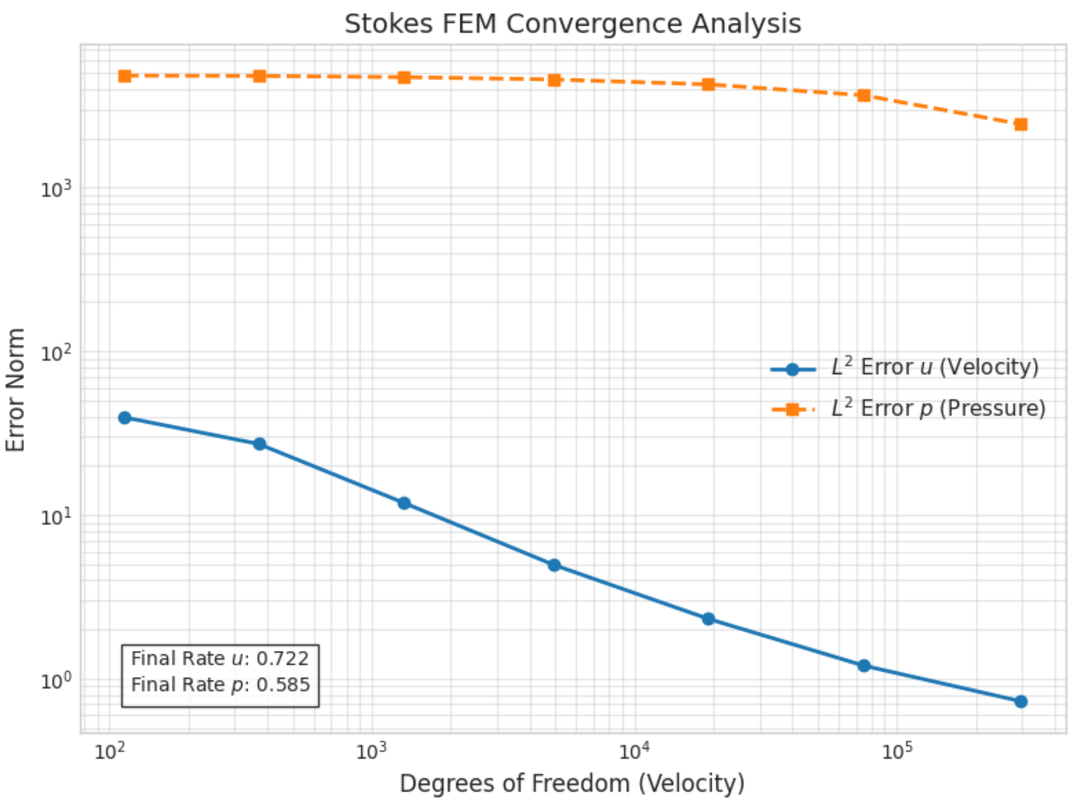


Figure 4: Log-log plot of the L^2 error norms for velocity and pressure against the velocity degrees of freedom.

Figure 4 visualizes the convergence behavior detailed in Table 2 by plotting the L^2 error norms against the velocity degrees of freedom on a logarithmic scale. The solid blue line demonstrates a steady, near-linear decrease in the velocity error, ultimately achieving a final convergence rate of $r_u = 0.722$. In contrast, the dashed orange line representing the pressure error starts at a significantly higher magnitude and exhibits a much flatter trajectory, finalizing at a rate of $r_p = 0.585$. This visual disparity emphasizes the slower convergence of the pressure field inherent to the P1-iso-P1 element pair, particularly when subjected to the geometric singularities of the Z-shaped domain.